
XAI_metrics

Maria Teresa Alba Rueda

Jun 16, 2026

1	xai_metrics.base package	1
1.1	Submodules	1
1.1.1	xai_metrics.base.base_explainer module	1
1.1.2	xai_metrics.base.base_metric module	2
1.1.3	xai_metrics.base.registry module	3
1.1.4	xai_metrics.base.types module	4
1.2	Module contents	4
2	xai_metrics.config package	5
2.1	Submodules	5
2.1.1	xai_metrics.config.config_controller module	5
2.2	Module contents	6
3	xai_metrics.metrics package	7
3.1	Subpackages	7
3.1.1	xai_metrics.metrics.complexity package	7
3.1.1.1	Submodules	7
3.1.1.2	Module contents	9
3.1.2	xai_metrics.metrics.faithfulness package	9
3.1.2.1	Submodules	9
3.1.2.2	Module contents	18
3.1.3	xai_metrics.metrics.fidelity package	18
3.1.3.1	Subpackages	18
3.1.3.2	Module contents	21
3.1.4	xai_metrics.metrics.robustness package	21
3.1.4.1	Submodules	21
3.1.4.2	Module contents	27
3.1.5	xai_metrics.metrics.sensitivity package	27
3.1.5.1	Submodules	27
3.1.5.2	Module contents	28
3.2	Submodules	28
3.2.1	xai_metrics.metrics.autodiscover module	28
3.3	Module contents	29
4	xai_metrics.reporting package	31
4.1	Submodules	31
4.1.1	xai_metrics.reporting.reporting module	31
4.2	Module contents	32
5	xai_metrics.runner package	33

5.1	Submodules	33
5.1.1	<code>xai_metrics.runner.runner</code> module	33
5.2	Module contents	34
Python Module Index		35
Index		37

XAI_METRICS.BASE PACKAGE

1.1 Submodules

1.1.1 xai_metrics.base.base_explainer module

class xai_metrics.base.base_explainer.BaseExplainer(*params=None*)

Bases: object

Base class for explanation methods.

Subclasses must implement `explain()`, which generates attribution values for a batch of inputs. Instances are callable and can also expose their explanation method as an `ExplainFunc` compatible with metrics that need to recompute explanations.

Parameters

params (*Mapping[str, Any] or None, optional*) – Configuration parameters used by the explanation method. A copy of the mapping is stored in `params`. If `None`, an empty dictionary is used.

NAME: `str = 'explainer'`

as_explain_func()

Return the explanation method as a metric-compatible function.

The returned bound method can be passed as an `explain_func` dependency to metrics that need to generate explanations for modified or perturbed inputs.

Returns

Bound `explain()` method of the current explainer instance.

Return type

`ExplainFunc`

explain(*model, inputs, targets=None, **kwargs*)

Generate explanations for the provided inputs.

Subclasses must override this method with the logic required by the corresponding explanation method.

Parameters

- **model** (`torch.nn.Module`) – Model whose predictions are to be explained.
- **inputs** (*Any*) – Input observation or batch of observations to explain.
- **targets** (*Any or None, optional*) – Target outputs, classes or labels for which explanations are generated. Their interpretation depends on the explainer.
- ****kwargs** (*Any*) – Additional arguments required by the concrete explanation method.

Returns

Attribution values generated for the provided inputs.

Return type

numpy.ndarray

Raises

NotImplementedError – Always raised by the base implementation. Subclasses must override this method.

1.1.2 xai_metrics.base.base_metric module

class xai_metrics.base.base_metric.**BaseMetric**(*context, params=None*)

Bases: object

Base class for all metric implementations.

Metric classes should inherit from this class, define their own NAME, and implement the [run\(\)](#) method.

Parameters

- **context** ([MetricContext](#)) – Shared context containing the model, test data, labels, observations, attribution values and optional device information.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific configuration parameters. If None, an empty dictionary is used.

NAME: str = 'metric'

run()

Execute the metric computation.

Subclasses must override this method with the actual metric logic.

Returns

Metric result returned by the concrete implementation.

Return type

Any

Raises

NotImplementedError – If the subclass does not implement this method.

class xai_metrics.base.base_metric.**MetricContext**(*model, X_test, y_test, observations, attributions, device=None*)

Bases: object

Shared context used by metric implementations.

This dataclass stores the model, test data, labels, selected observations, attribution values, and optional extra information required by metric classes.

Variables

- **model** (*torch.nn.Module*) – Model evaluated by the metrics.
- **X_test** (*pandas.DataFrame*) – Test input data used for metric computation.
- **y_test** (*pandas.Series*) – Test labels associated with X_test.
- **observations** (*Any*) – Identifiers of the observations explained by the attribution matrix. These values usually correspond to indexes in X_test and y_test.

- **attributions** (*numpy.ndarray*) – Attribution values for the selected observations. Each row usually corresponds to one observation and each column to one feature.
- **device** (*str* or *None*) – Device where the model is placed, such as "cpu" or "cuda". If *None*, no explicit device was configured.

Parameters

- **model** (*Module*)
- **X_test** (*DataFrame*)
- **y_test** (*Series*)
- **observations** (*Any*)
- **attributions** (*ndarray*)
- **device** (*str* | *None*)

device: *str* | *None* = *None*

exception `xai_metrics.base.base_metric.MetricSkipped`

Bases: `Exception`

Exception raised when a metric cannot be computed.

This exception should be used when a metric is not applicable to the current context, model, data, or attribution values.

1.1.3 xai_metrics.base.registry module

`xai_metrics.base.registry.build_metrics_from_config(metrics_cfg, context, dependencies=None)`

Build metric instances from a configuration list.

Each metric configuration must contain a **name** field matching a registered metric. It may also contain a **params** field with metric-specific parameters. Additional dependencies are injected only if their names are accepted by the metric class constructor.

Parameters

- **metrics_cfg** (*List[Mapping[str, Any]]*) – List of metric configuration dictionaries. Each dictionary must contain the metric name and may contain **params**.
- **context** (*MetricContext*) – Shared metric evaluation context passed to every metric instance.
- **dependencies** (*Mapping[str, Any]* or *None, optional*) – Optional dependency objects that can be injected into metric constructors. Only dependencies whose names appear in the constructor signature are passed.

Returns

List of instantiated metric objects.

Return type

List[*BaseMetric*]

Raises

- **ValueError** – If a metric name from `metrics_cfg` is not registered.
- **KeyError** – If a metric configuration does not contain the required **name** field.

`xai_metrics.base.registry.list_metrics()`

Return the names of the registered metrics.

Returns

Sorted list containing the names of all metrics currently registered in METRIC_REGISTRY.

Return type

List[str]

`xai_metrics.base.registry.register_metric(cls)`

Register a metric class in the global metric registry.

The metric is registered using its NAME attribute. If the class does not define NAME, the class name is used instead.

Parameters

cls (Type[BaseMetric]) – Metric class to register.

Returns

The same metric class passed as input. This allows the function to be used as a decorator.

Return type

Type[BaseMetric]

Raises

ValueError – If another metric with the same name is already registered.

Examples

```
>>> @register_metric
... class MyMetric(BaseMetric):
...     NAME = "my_metric"
...     def run(self):
...         return 0.0
```

1.1.4 xai_metrics.base.types module

`type xai_metrics.base.types.ExplainFunc = Callable[[Module, Any, Any | None], ndarray]`

1.2 Module contents

XAI_METRICS.CONFIG PACKAGE

2.1 Submodules

2.1.1 xai_metrics.config.config_controller module

```
class xai_metrics.config.config_controller.ConfigController(config='config.yaml',  
                                                         model_loader=None)
```

Bases: object

Controller for loading configuration files and building metric contexts.

The controller can load a configuration from a dictionary-like object or from a YAML file. It supports both direct context definitions and automatic discovery of contexts from dataset, model and attribution directories.

Parameters

- **config** (*Mapping[str, Any], str or pathlib.Path, default="config.yaml"*) – Configuration source. If a mapping is provided, it is copied directly. If a path is provided, the YAML file is loaded. The default value loads the config.yaml file located next to this module.
- **model_loader** (*Callable or None, optional*) – Function used to load models from disk. The function must accept a model path as input and return the loaded model object. If None, [default_model_loader\(\)](#) is used.

```
build_context(ctx_cfg=None)
```

Build a metric evaluation context.

The context configuration must contain paths to the model, test data, test labels and attribution file. The method loads these objects, validates their indexes, converts attribution values to a NumPy array, optionally moves the model to the configured device, and returns both the `MetricContext` and its identifying metadata.

Parameters

ctx_cfg (*Mapping[str, Any] or None, optional*) – Context configuration to use. If None, the context section of the loaded configuration is used. The configuration must contain `model_path`, `X_test_path`, `y_test_path` and `attributions_path`. It must also contain the metadata fields `dataset_name`, `model_name` and `xai_method_name`. It may optionally contain `device`.

Returns

Metric context and metadata dictionary. The metadata contains `dataset_name`, `model_name` and `xai_method_name`.

Return type

Tuple[[MetricContext](#), Dict[str, Any]]

Raises

- **ValueError** – If the `context` section is missing, required paths are missing, metadata is incomplete, or indexes are inconsistent.
- **TypeError** – If the loaded model is not an instance of `torch.nn.Module`.
- **RuntimeError** – If a CUDA device is requested but CUDA is not available.

Warns

UserWarning – If `y_test` contains more than one column. In that case, only the first column is used.

build_contexts()

Build all metric evaluation contexts defined by the configuration.

The method first obtains all context configurations through `_iter_context_configs()` and then builds each corresponding `MetricContext`.

Returns

List of metric contexts together with their metadata.

Return type

List[Tuple[*MetricContext*, Dict[str, Any]]]

get_metrics_config()

Return the metrics configuration section.

Returns

List of metric configurations. If the section is not defined, an empty list is returned.

Return type

List

xai_metrics.config.config_controller.default_model_loader(model_path)

Load a model from disk.

Files with `.pkl` or `.pickle` extensions are loaded with `pickle`. Files with `.joblib` or `.jll` extensions are loaded with `joblib`. Any other file extension is loaded with `torch.load()` using `weights_only=False`.

Parameters

model_path (*str* or *pathlib.Path*) – Path to the saved model file.

Returns

Loaded model object.

Return type

Any

2.2 Module contents

XAI_METRICS.METRICS PACKAGE

3.1 Subpackages

3.1.1 xai_metrics.metrics.complexity package

3.1.1.1 Submodules

xai_metrics.metrics.complexity.complexity_metric module

class xai_metrics.metrics.complexity.complexity_metric.**Complexity**(*context*, *params=None*)

Bases: *BaseMetric*

Quantus Complexity metric.

This metric measures explanation complexity using the entropy of the attribution distribution. Quantus takes the absolute attribution values and expresses each feature contribution as a fraction of the total attribution magnitude.

Higher scores indicate that importance is distributed across more features and therefore corresponds to more complex explanations. Lower scores indicate that importance is concentrated on fewer features.

For an explanation with n features, the maximum entropy is approximately $\log(n)$ when importance is distributed uniformly. The scores returned by this wrapper are not divided by this maximum value.

The metric is based on the Complexity metric proposed by Bhatt et al. (2020) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Quantus also applies the absolute-value operation to the attributions because its `abs` parameter is not exposed by this wrapper and defaults to `True`.

NAME: `str = 'Complexity'`

run()

Compute the Complexity metric.

The method passes the selected input data, labels and attribution values to `quantus.Complexity`. Quantus flattens each explanation, takes its absolute values and computes the entropy of the fractional feature contributions.

If all attribution values are negative, this wrapper converts them to absolute values before calling Quantus. The model is set to training mode following the current wrapper implementation, although the metric itself depends only on the attribution values.

Returns

Complexity score for each evaluated observation. Lower values indicate that importance is concentrated on fewer features, while higher values indicate that it is more widely distributed. Scores are not normalised by the theoretical maximum $\log(n)$.

Return type

List[float]

xai_metrics.metrics.complexity.sparseness module

class `xai_metrics.metrics.complexity.sparseness.Sparseness`(*context*, *params=None*)

Bases: `BaseMetric`

Quantus Sparseness metric.

This metric measures how concentrated the attribution magnitude is across the input features using the Gini index. Quantus applies the absolute-value operation to the attributions before computing the score.

Higher scores indicate that importance is concentrated on a smaller subset of features and therefore correspond to sparser explanations. Lower scores indicate a more uniform distribution of attribution magnitude.

The metric is based on the Sparseness metric proposed by Chalasani et al. (2020) and implemented in Quantus.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Quantus applies the absolute-value operation to the attributions because its `abs` parameter is not exposed by this wrapper and defaults to `True`.

NAME: `str = 'Sparseness'`

run()

Compute the Sparseness metric.

The method passes the selected input data, labels and attribution values to `quantus.Sparseness`. Quantus flattens each explanation, takes the absolute attribution values, sorts them and computes their Gini index.

If all attribution values are negative, this wrapper converts them to absolute values before calling Quantus. The model is set to training mode following the current wrapper implementation, although the metric itself depends only on the attribution values.

Returns

Sparseness score for each evaluated observation. Higher values indicate that attribution magnitude is concentrated on fewer features, while lower values indicate a more uniform distribution.

Return type

List[float]

3.1.1.2 Module contents

3.1.2 xai_metrics.metrics.faithfulness package

3.1.2.1 Submodules

xai_metrics.metrics.faithfulness.consistency module

class xai_metrics.metrics.faithfulness.consistency.**Consistency**(context, params=None)

Bases: *BaseMetric*

Quantus Consistency metric.

This metric evaluates whether observations with equivalent explanations exhibit consistent model behaviour. Continuous attribution vectors are discretised by Quantus and observations sharing the same discrete representation are grouped together. For each observation, the metric computes the proportion of other observations in its group that receive the same predicted class.

Higher scores indicate stronger agreement between similar explanations and model predictions.

The metric is based on the Consistency metric proposed by Dasgupta et al. (2022) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **abs** : bool, optional Whether to use absolute attribution values before computing the metric. If all attribution values are negative and this parameter is `True`, their absolute values are used. If it is `False`, the metric is skipped. The default value is `True`.
 - **normalise** : bool, optional Whether to normalise the attribution values before discretising the explanations. The default value is `True`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default discretisation and normalisation functions provided by Quantus. The default discretisation function is `top_n_sign`.

NAME: `str = 'Consistency'`

run()

Compute the Consistency metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.Consistency`. Quantus discretises the explanations, predicts the class of each observation and compares predictions among observations with equivalent discrete explanations.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Consistency score for each evaluated observation. Higher values indicate that observations with the same discretised explanation more frequently receive the same predicted class.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is False.

xai_metrics.metrics.faithfulness.faithfulness module

```
class xai_metrics.metrics.faithfulness.faithfulness.Faithfulness(context, params=None)
```

Bases: *BaseMetric*

AIX360 Faithfulness metric.

This metric evaluates whether feature attribution values reflect the influence of the corresponding features on the model prediction. For each observation, every feature is individually replaced with a baseline value and the probability assigned to the original predicted class is recorded. The score is the negative Pearson correlation between the attribution values and these probabilities.

Higher scores indicate stronger agreement between feature importance and the effect of replacing features with their baseline values.

The wrapped AIX360 implementation requires a classification model exposing a `predict_proba` method.

The metric is based on the faithfulness criterion proposed by Alvarez-Melis and Jaakkola (2018) and implemented in AIX360.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. The model must implement `predict_proba`.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `base_values` : array-like or None, optional Explicit baseline values used to replace feature values. If provided, this takes priority over `base_func` and `base_strategy`.
 - `base_strategy` : str, optional Strategy used to compute baseline values from `X_test` when `base_values` and `base_func` are not provided. Supported values are "mean", "median" and "zero". The default value is "mean".

If None, an empty dictionary is used.

NAME: `str = 'Faithfulness'`

run()

Compute the Faithfulness metric.

The method resolves a common baseline vector from the complete test dataset and evaluates each selected observation independently using `aix360.metrics.faithfulness_metric()`.

For each observation, AIX360 determines the predicted class, replaces each feature individually with its baseline value and records the resulting probability for that class. The returned score is the negative Pearson correlation between these probabilities and the attribution values.

Returns

Faithfulness score for each evaluated observation. Higher values indicate that features with larger attribution values produce greater decreases in the predicted class probability when replaced.

Return type

List[float]

Raises

- **ValueError** – If `base_strategy` is not "mean", "median" or "zero".
- **AttributeError** – If the model does not implement the `predict_proba` method required by AIX360.

Notes

The score may be `nan` when the attribution values or the probabilities obtained after feature replacement have zero variance.

xai_metrics.metrics.faithfulness.faithfulness_estimate module

```
class xai_metrics.metrics.faithfulness.faithfulness_estimate.FaithfulnessEstimate(context,
                                                                                params=None)
```

Bases: *BaseMetric*

Quantus Faithfulness Estimate metric.

This metric evaluates whether features with higher attribution values produce larger changes in the model output when they are perturbed. Features are perturbed in groups ordered by attribution importance, and the metric compares the sum of their attributions with the corresponding drop in the target output.

This wrapper uses a safe Pearson correlation function that returns `0.0` when either of the compared vectors has zero variance, avoiding undefined correlation values.

Higher scores indicate stronger agreement between attribution importance and changes in the model output.

The metric is based on the Faithfulness Estimate proposed by Alvarez-Melis and Jaakkola (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `features_in_step`: int, optional Number of features perturbed at each step. The default value is 1.

- `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
- `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
- `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

Notes

This wrapper uses its internal `_safe_pearson()` method as the similarity function. The default normalisation and perturbation functions provided by Quantus are used.

NAME: `str = 'FaithfulnessEstimate'`

`run()`

Compute the Faithfulness Estimate metric.

The method selects the observations defined in the metric context and passes their input data, labels and attribution values to `quantus.FaithfulnessEstimate`. Features are perturbed in groups, and the internal safe Pearson correlation function compares attribution sums with the resulting target-output drops.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed.

Returns

Faithfulness Estimate score for each evaluated observation. Higher values indicate stronger agreement between attribution importance and model output changes after perturbation.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

`xai_metrics.metrics.faithfulness.monotonicity` module

class `xai_metrics.metrics.faithfulness.monotonicity.Monotonicity(context, params=None)`

Bases: *`BaseMetric`*

Quantus Monotonicity metric.

This metric evaluates whether the target model output increases monotonically as features are progressively introduced from a baseline input. Features are processed in increasing order of attribution value and may be introduced individually or in groups.

For each observation, the metric returns `True` when the sequence of target outputs is monotonically non-decreasing and `False` otherwise.

The metric is based on the Monotonicity metric proposed by Arya et al. (2019) and the monotonic attribute functions described by Luss et al. (2019), as implemented in Quantus.

Parameters

- **context** (*`MetricContext`*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.

- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **features_in_step** : int, optional Number of features added at each perturbation step. The default value is 1.
 - **abs** : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is True.
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is True.
 - **perturb_baseline** : str, optional Baseline value used to initialise the perturbed input. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If None, an empty dictionary is used.

Notes

The wrapper uses the default normalisation and perturbation functions provided by Quantus.

NAME: str = 'Monotonicity'

run()

Compute the Monotonicity metric.

The method passes the selected input data, target labels and attribution values to `quantus.Monotonicity`. For each observation, Quantus starts from a baseline input, processes features in increasing order of attribution value and evaluates the target model output after each group is introduced.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Monotonicity result for each evaluated observation. True indicates that the target output is monotonically non-decreasing across the feature-introduction steps.

Return type

List[bool]

Raises

MetricSkipped – If all attribution values are negative and `abs` is False.

xai_metrics.metrics.faithfulness.monotonicity_correlation module

class xai_metrics.metrics.faithfulness.monotonicity_correlation.**MonotonicityCorrelation**(*context*,
params=None)

Bases: *BaseMetric*

Quantus Monotonicity Correlation metric.

This metric evaluates whether feature attribution values are monotonically related to the uncertainty caused by perturbing the corresponding features. Features are grouped in increasing order of attribution, and each group is perturbed repeatedly to estimate its effect on the target model output. The score is the Spearman correlation between the attribution sums and the estimated output variations.

This wrapper uses a safe Spearman correlation that returns 0.0 when either input vector has zero variance.

Higher scores indicate a stronger positive relationship between feature importance and the uncertainty caused by perturbing those features.

The metric is based on the Monotonicity Correlation metric proposed by Nguyen and Rodríguez Martínez (2020) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `eps` : float, optional Threshold used when computing the inverse prediction factor. The default value is `1e-5`.
 - `nr_samples` : int, optional Number of perturbation samples generated for each feature group. The default value is `100`.
 - `features_in_step` : int, optional Number of features perturbed at each step. The default value is `1`.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "uniform".

If `None`, an empty dictionary is used.

Notes

The wrapper uses `_safe_spearman()` as the similarity function and the default normalisation and perturbation functions provided by Quantus.

NAME: `str = 'MonotonicityCorrelation'`

`run()`

Compute the Monotonicity Correlation metric.

The method passes the selected inputs, labels and attribution values to `quantus.MonotonicityCorrelation`. Quantus perturbs groups of features repeatedly, estimates their relative effect on the target output and compares those estimates with the corresponding attribution sums using `_safe_spearman()`.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Monotonicity Correlation score for each evaluated observation. Higher values indicate a stronger positive relationship between attribution importance and output variation.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.fairness.monotonicity_metric module

class xai_metrics.metrics.fairness.monotonicity_metric.**MonotonicityMetric**(context,
params=None)

Bases: *BaseMetric*

AIX360 Monotonicity metric.

This metric evaluates whether the probability assigned to the originally predicted class increases monotonically as features are progressively restored from a baseline input.

For each observation, features are restored in increasing order of their attribution values. The metric returns True when the resulting sequence of predicted-class probabilities is monotonically non-decreasing.

The wrapped AIX360 implementation requires a classification model exposing a `predict_proba` method.

The metric is based on the monotonicity criterion described by Luss et al. (2019) and implemented in AIX360.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values. The model must implement `predict_proba`.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `base_values` : array-like or None, optional Explicit baseline values used as the initial feature values. If provided, this takes priority over `base_func` and `base_strategy`.
 - `base_strategy` : str, optional Strategy used to compute baseline values from `X_test` when `base_values` and `base_func` are not provided. Supported values are "mean", "median" and "zero". The default value is "mean".

If None, an empty dictionary is used.

NAME: str = 'MonotonicityMetric'

run()

Compute the Monotonicity metric.

The method resolves a common baseline vector from the complete test dataset and evaluates each selected observation independently using `aix360.metrics.monotonicity_metric()`.

For each observation, AIX360 determines the class predicted for the original input. Starting from the baseline, it restores features cumulatively in increasing order of attribution value and evaluates the probability assigned to that class after each restoration.

Returns

Monotonicity result for each evaluated observation. True indicates that the predicted-class probability never decreases as features are progressively restored.

Return type

List[bool]

Raises

- **ValueError** – If `base_strategy` is not "mean", "median" or "zero".
- **AttributeError** – If the model does not implement the `predict_proba` method required by AIX360.

xai_metrics.metrics.fairness.sensitivity_n module

class xai_metrics.metrics.fairness.sensitivity_n.SensitivityN(context, params=None)

Bases: *BaseMetric*

Quantus Sensitivity-N metric.

This metric evaluates the agreement between feature attribution values and the variation in the target model output caused by perturbing the corresponding features.

Quantus orders features by decreasing attribution value and progressively perturbs them in groups. At each step, it compares the target-output change with the attribution sum of the processed feature group using Pearson correlation across the evaluated observations.

The number of perturbation steps is limited by `n_max_percentage`. Higher correlation values indicate stronger agreement between the explanation and the model behaviour.

The metric is based on the Sensitivity-N test proposed by Ancona et al. (2018) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `n_max_percentage` : float, optional Maximum percentage of features to evaluate. The default value is 0.8.
 - `features_in_step` : int, optional Number of features perturbed at each step. The default value is 1.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is False.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is True.
 - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If None, an empty dictionary is used.

Notes

The wrapper uses the default Pearson correlation, normalisation and baseline-replacement functions provided by Quantus. Quantus aggregates the step-wise correlations by default.

NAME: str = 'SensitivityN'

run()

Compute the Sensitivity-N metric.

The method passes the selected input data, target labels and attribution values to `quantus.SensitivityN`. Quantus progressively perturbs feature groups ordered by decreasing attribution value and computes the Pearson correlation between their attribution sums and the resulting target-output changes.

Only the perturbation steps covered by `n_max_percentage` are evaluated. If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Sensitivity-N result returned by Quantus. Higher values indicate stronger agreement between attribution values and target-output changes. With the default Quantus configuration, the step-wise correlations are aggregated.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and abs is False.

xai_metrics.metrics.faithfulness.sufficiency module

class xai_metrics.metrics.faithfulness.sufficiency.**Sufficiency**(context, params=None)

Bases: *BaseMetric*

Quantus Sufficiency metric.

This metric evaluates whether observations with similar explanations receive the same model prediction. Quantus computes the pairwise distances between attribution vectors and considers two explanations similar when their distance is less than or equal to **threshold**.

For each observation, the score is the proportion of other observations with similar explanations that receive the same predicted class. A score of 0.0 is returned when no other explanation satisfies the distance threshold.

Higher scores indicate stronger agreement between similar explanations and model predictions. Since explanations are compared within the evaluated batch, the results depend on the selected observations.

The metric is based on the Sufficiency test proposed by Dasgupta et al. (2022) and implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, **X_test**, **y_test**, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - **threshold** : float, optional Maximum distance between two attribution vectors for their explanations to be considered similar. The default value is 0.6.
 - **distance_func** : str, optional Distance function used to compare attribution vectors. The value is passed to Quantus and is typically a valid SciPy distance name, such as "seuclidean". The default value is "seuclidean".
 - **abs** : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is True.
 - **normalise** : bool, optional Whether to normalise the attribution values before computing the metric. The default value is True.

If None, an empty dictionary is used.

Notes

The wrapper uses the default normalisation function provided by Quantus. Scores depend on the observations evaluated in the same call, since these observations define the explanation neighbourhoods.

NAME: str = 'Sufficiency'

run()

Compute the Sufficiency metric.

The method passes the selected input data, labels and attribution values to `quantus.Sufficiency`. Quantus compares the attribution vectors, identifies observations with similar explanations and computes the proportion that receive the same predicted class. Self-comparisons are excluded.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to evaluation mode before the computation.

Returns

Sufficiency score for each evaluated observation. Scores range from 0.0 to 1.0. Higher values indicate that observations with similar explanations more frequently receive the same predicted class.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

3.1.2.2 Module contents**3.1.3 xai_metrics.metrics.fidelity package****3.1.3.1 Subpackages****xai_metrics.metrics.fidelity.completeness package****Submodules****xai_metrics.metrics.fidelity.completeness.completeness_metric module**

```
class xai_metrics.metrics.fidelity.completeness.completeness_metric.Completeness(context,
                                                                                    params=None,
                                                                                    nor-
                                                                                    malise_func=None,
                                                                                    nor-
                                                                                    malise_func_kwargs=None,
                                                                                    out-
                                                                                    put_func=None,
                                                                                    per-
                                                                                    turb_func=None,
                                                                                    per-
                                                                                    turb_func_kwargs=None)
```

Bases: *BaseMetric*

Quantus Completeness metric.

This metric evaluates whether the total attribution assigned to an input is consistent with the difference between the model output at the original input and the model output at a baseline input. This property is also known as summation to delta or conservation.

The metric is based on the Completeness property from Sundararajan et al. (2017), the Summation to Delta property from Shrikumar et al. (2017), and the Conservation property discussed by Montavon et al. (2018), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used to create the reference input. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

- **normalise_func** (*Callable[..., numpy.ndarray] or None, optional*) – Custom normalisation function passed to Quantus. The function must accept the attribution array as its first argument and may accept additional keyword arguments from `normalise_func_kwargs`. If `None`, Quantus uses its default normalisation behaviour when `normalise=True`.
- **normalise_func_kwargs** (*Dict[str, Any] or None, optional*) – Keyword arguments passed to `normalise_func` when normalisation is enabled. If `None`, no additional keyword arguments are passed.
- **output_func** (*Callable[..., numpy.ndarray] or None, optional*) – Function applied to the difference between the model output at the original input and the model output at the baseline input before comparing it with the attribution sum. If `None`, Quantus uses its default identity function.
- **perturb_func** (*Callable[..., numpy.ndarray] or None, optional*) – Perturbation function passed to Quantus. The function must be compatible with Quantus perturbation functions, accepting at least an input array and feature indices, and returning the perturbed array. If `None`, Quantus uses its default perturbation function.
- **perturb_func_kwargs** (*Dict[str, Any] or None, optional*) – Keyword arguments passed to `perturb_func`. If `None`, no additional keyword arguments are passed.

NAME: `str = 'Completeness'`

run()

Compute the Completeness metric.

The method selects the observations defined in the metric context, retrieves their input data, labels and attribution values, and passes them to `quantus.Completeness`. The model is set to evaluation mode before computing the metric.

Returns

Completeness result for each evaluated observation. `True` indicates that the sum of the attribution values matches the transformed difference between the model output at the input and the model output at the baseline input.

Return type

`List[bool]`

Raises

MetricSkipped – If all attribution values are negative, since the metric is skipped for that attribution configuration.

Module contents

xai_metrics.metrics.fidelity.soundness package

Submodules

xai_metrics.metrics.fidelity.soundness.non_sensitivity module

```
class xai_metrics.metrics.fidelity.soundness.non_sensitivity.NonSensitivity(context,
                                                                           params=None,
                                                                           normalise_func=None,
                                                                           normalise_func_kwargs=None,
                                                                           perturb_func=None,
                                                                           perturb_func_kwargs=None)
```

Bases: [BaseMetric](#)

Quantus Non-Sensitivity metric.

This metric evaluates whether near-zero attribution values are assigned only to features that do not significantly affect the model output. For each observation, the metric compares features with attribution values below a threshold against features whose perturbation produces negligible change in the model prediction.

The metric is based on the Non-Sensitivity metric discussed by Nguyen and Rodríguez Martínez (2020), Ancona et al. (2019), and Montavon et al. (2018), as implemented in Quantus.

Parameters

- **context** ([MetricContext](#)) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations and attribution values.
- **params** ([Mapping\[str, Any\]](#) or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `eps` : float, optional Threshold used to decide whether an attribution value is considered negligible and whether a prediction change is considered insignificant. The default value is `1e-5`.
 - `features_in_step` : int, optional Number of features perturbed at each step. The default value is `1`.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `True`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_baseline` : str, optional Baseline value used when perturbing features. Supported values depend on the Quantus perturbation function. Common values are "black", "white", "mean", "random" and "uniform". The default value is "black".

If `None`, an empty dictionary is used.

- **normalise_func** ([Callable\[...\]](#), [numpy.ndarray](#) or `None`, optional) – Custom normalisation function passed to Quantus. The function must accept the attribution array as its first argument and may accept additional keyword arguments from `normalise_func_kwargs`. If `None`, Quantus uses its default normalisation behaviour when `normalise=True`.

- **normalise_func_kwargs** (*Dict[str, Any] or None, optional*) – Keyword arguments passed to `normalise_func` when normalisation is enabled. If `None`, no additional keyword arguments are passed.
- **perturb_func** (*Callable[..., numpy.ndarray] or None, optional*) – Perturbation function passed to Quantus. The function must be compatible with Quantus perturbation functions, accepting at least an input array and feature indices, and returning the perturbed array. If `None`, Quantus uses its default perturbation function.
- **perturb_func_kwargs** (*Dict[str, Any] or None, optional*) – Keyword arguments passed to `perturb_func`. If `None`, no additional keyword arguments are passed.

NAME: `str = 'NonSensitivity'`

run()

Compute the Non-Sensitivity metric.

The method selects the observations defined in the metric context, retrieves their input data, labels and attribution values, and passes them to `quantus.NonSensitivity`. The model is set to evaluation mode before computing the metric.

Returns

Non-Sensitivity score for each evaluated observation. Lower values indicate better agreement between near-zero attributions and features that do not significantly affect the model output.

Return type

List[float]

Raises

MetricSkipped – If all attribution values are negative, since the metric is skipped for that attribution configuration.

Module contents

3.1.3.2 Module contents

3.1.4 xai_metrics.metrics.robustness package

3.1.4.1 Submodules

`xai_metrics.metrics.robustness.local_lipschitz_estimate` module

```
class xai_metrics.metrics.robustness.local_lipschitz_estimate.LocalLipschitzEstimate(context,
                                                                                       ex-
                                                                                       plain_func,
                                                                                       params=None)
```

Bases: *`BaseMetric`*

Quantus Local Lipschitz Estimate metric.

This metric evaluates the local stability of an explanation by comparing changes in the explanation with changes in the corresponding input. For each observation, Quantus generates neighbouring inputs using Gaussian noise, recomputes their explanations and calculates the ratio between the explanation distance and the input distance. The score is the maximum ratio obtained across the sampled neighbours.

Lower scores indicate that nearby inputs receive similar explanations and therefore correspond to more locally robust explanations.

The metric is based on the Local Lipschitz Estimate proposed by Alvarez-Melis and Jaakkola (2018) and implemented in Quantus.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (`ExplainFunc`) – Function used to generate explanations for perturbed inputs. The function must accept the model, a batch of inputs and optionally a batch of labels or targets, and must return a NumPy array containing the generated attributions.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `True`.
 - `perturb_mean` : float, optional Mean of the Gaussian noise used for perturbations. The default value is 0.0.
 - `perturb_std` : float, optional Standard deviation of the Gaussian noise used for perturbations. The default value is 0.1.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: the Lipschitz constant as the comparison function, Euclidean distance for the numerator and denominator, the default attribution normalisation function and Gaussian-noise perturbations.

The score is a finite-sample estimate based on the generated neighbours, rather than the exact maximum over the complete local neighbourhood.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (`MetricContext`)
- **explain_func** (`ExplainFunc`)
- **params** (`Mapping[str, Any]` | `None`)

NAME: `str = 'LocalLipschitzEstimate'`

run()

Compute the Local Lipschitz Estimate metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.LocalLipschitzEstimate`. Quantus samples neighbouring inputs, recomputes their explanations and returns the maximum ratio between explanation and input changes for each observation.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to training mode and the device stored in the context is forwarded to Quantus.

Returns

Local Lipschitz Estimate score for each evaluated observation. Lower values indicate greater local explanation stability.

Return type

List[float]

Raises***MetricSkipped*** – If all attribution values are negative and `abs` is `False`.**xai_metrics.metrics.robustness.max_sensitivity module**

```
class xai_metrics.metrics.robustness.max_sensitivity.MaxSensitivity(context, explain_func,
                                                                params=None)
```

Bases: *BaseMetric*

Quantus Max-Sensitivity metric.

This metric evaluates explanation robustness by measuring the largest relative change in an explanation under small input perturbations. For each observation, Quantus generates several perturbed inputs, recomputes their explanations and compares them with the original explanation. The score is the maximum ratio between the norm of the explanation change and the norm of the original explanation.

Lower scores indicate more robust explanations, while higher scores indicate greater sensitivity to input perturbations.

The metric is based on Max-Sensitivity proposed by Yeh et al. (2019) and discussed by Bhatt et al. (2020), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.
 - `lower_bound` : float, optional Lower bound of the uniform noise used for perturbations. The default value is 0.2.
 - `upper_bound` : float or None, optional Upper bound of the uniform noise used for perturbations. If None, Quantus uses its default behaviour. The default value is None.

If None, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: element-wise difference to compare explanations, the Frobenius norm for the numerator and denominator, the default normalisation function and uniform-noise perturbations.

Raises**ValueError** – If `explain_func` is not provided.

Parameters

- **context** (`MetricContext`)
- **explain_func** (`ExplainFunc`)
- **params** (`Mapping[str, Any] | None`)

NAME: `str = 'MaxSensitivity'`

run()

Compute the Max-Sensitivity metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.MaxSensitivity`. Quantus repeatedly perturbs each input, recomputes its explanation and returns the maximum relative explanation change across the configured perturbations.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to training mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Max-Sensitivity score for each evaluated observation. Lower values indicate explanations that are more robust to small random input perturbations.

Return type

`List[float]`

Raises

`MetricSkipped` – If all attribution values are negative and `abs` is `False`.

xai_metrics.metrics.robustness.relative_input_stability module

```
class xai_metrics.metrics.robustness.relative_input_stability.RelativeInputStability(context,
                                                                                      ex-
                                                                                      plain_func,
                                                                                      params=None)
```

Bases: *`BaseMetric`*

Quantus Relative Input Stability metric.

This metric evaluates explanation robustness by comparing the relative change in an explanation with the relative change in its input. For each observation, Quantus generates several perturbed inputs, recomputes their explanations and returns the maximum ratio between both relative changes.

Lower scores indicate more stable explanations, while higher scores indicate greater sensitivity to input perturbations.

The metric is based on Relative Input Stability proposed by Agarwal et al. (2022), as implemented in Quantus.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (`ExplainFunc`) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (`Mapping[str, Any] or None, optional`) – Metric-specific parameters. Supported keys are:

- `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
- `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
- `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.

If `None`, an empty dictionary is used.

Notes

Quantus uses uniform-noise perturbations with an upper bound of 0.2. When normalisation is enabled, the default function is `normalise_by_average_second_moment_estimate`.

A constant of 1e-6 is used to avoid division by zero. By default, perturbations that change the model prediction produce nan scores.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- `context` (`MetricContext`)
- `explain_func` (`ExplainFunc`)
- `params` (`Mapping[str, Any] | None`)

NAME: `str = 'RelativeInputStability'`

`run()`

Compute the Relative Input Stability metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.RelativeInputStability`. Quantus perturbs each input, recomputes its explanation and returns the maximum ratio between the relative explanation change and the relative input change.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Relative Input Stability score for each evaluated observation. Lower values indicate more stable explanations. A score may be nan when a perturbation changes the model prediction.

Return type

`list[float]`

Raises

`MetricSkipped` – If all attribution values are negative and `abs` is `False`.

`xai_metrics.metrics.robustness.relative_output_stability` module

```
class xai_metrics.metrics.robustness.relative_output_stability.RelativeOutputStability(context,
                                                                                       ex-
                                                                                       plain_func,
                                                                                       params=None)
```

Bases: *BaseMetric*

Quantus Relative Output Stability metric.

This metric evaluates explanation robustness by comparing the relative change in an explanation with the corresponding change in the model output logits. For each observation, Quantus perturbs the input, recomputes its explanation and returns the maximum ratio obtained across the sampled perturbations.

Lower scores indicate that explanations change less relative to changes in the model output and are therefore considered more stable.

The metric is based on Relative Output Stability proposed by Agarwal et al. (2022), as implemented in Quantus.

Parameters

- **context** (*MetricContext*) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (*ExplainFunc*) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (*Mapping[str, Any] or None, optional*) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.
 - `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is False.
 - `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is False.

If None, an empty dictionary is used.

Notes

Quantus uses uniform-noise perturbations with an upper bound of 0.2. When normalisation is enabled, the default function is `normalise_by_average_second_moment_estimate`.

A constant of 1e-6 is used to avoid division by zero. By default, perturbations that change the predicted class produce nan scores.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- **context** (*MetricContext*)
- **explain_func** (*ExplainFunc*)
- **params** (*Mapping[str, Any] | None*)

NAME: `str = 'RelativeOutputStability'`

`run()`

Compute the Relative Output Stability metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.RelativeOutputStability`. Quantus repeatedly perturbs each input and returns the maxi-

ratio between the relative explanation change and the distance between the original and perturbed output-logit vectors.

If all attribution values are negative, their treatment depends on the `abs` parameter. Their absolute values are used when `abs=True`; otherwise, the metric is skipped.

The model is set to evaluation mode before the metric is computed. The device stored in the metric context is forwarded to Quantus.

Returns

Relative Output Stability score for each evaluated observation. Lower values indicate more stable explanations. A score may be `nan` when a perturbation changes the predicted class.

Return type

`list[float]`

Raises

`MetricSkipped` – If all attribution values are negative and `abs` is `False`.

3.1.4.2 Module contents

3.1.5 xai_metrics.metrics.sensitivity package

3.1.5.1 Submodules

xai_metrics.metrics.sensitivity.avg_sensitivity module

class `xai_metrics.metrics.sensitivity.avg_sensitivity.AvgSensitivity`(*context*, *explain_func*, *params=None*)

Bases: `BaseMetric`

Quantus Average Sensitivity metric.

This metric evaluates explanation robustness by measuring the average relative change in an explanation when small random perturbations are applied to its input. For each observation, Quantus generates several perturbed inputs, recomputes their explanations using `explain_func` and compares them with the original explanation.

The sensitivity of each perturbation is computed from the norm of the explanation difference relative to the norm of the original explanation. The final score is the average over all sampled perturbations.

Lower scores indicate more robust explanations, whereas higher scores indicate greater sensitivity to small input changes.

The metric is based on Average Sensitivity proposed by Yeh et al. (2019) and discussed by Bhatt et al. (2020), as implemented in Quantus.

Initialize the Average Sensitivity metric.

Parameters

- **context** (`MetricContext`) – Shared metric evaluation context. It must contain the model, `X_test`, `y_test`, selected observations, attribution values and optional device information.
- **explain_func** (`ExplainFunc`) – Function used to generate explanations for perturbed inputs. The function must be compatible with Quantus explanation functions and return a NumPy array containing the generated attributions.
- **params** (`Mapping[str, Any]` or `None`, optional) – Metric-specific parameters. Supported keys are:
 - `nr_samples` : int, optional Number of perturbed samples generated for each observation. The default value is 200.

- `abs` : bool, optional Whether to apply the absolute value operation to the attribution values before computing the metric. The default value is `False`.
- `normalise` : bool, optional Whether to normalise the attribution values before computing the metric. The default value is `False`.
- `lower_bound` : float, optional Lower bound of the uniform noise used for perturbations. The default value is `0.2`.
- `upper_bound` : float or `None`, optional Upper bound of the uniform noise used for perturbations. If `None`, Quantus uses its default behaviour. The default value is `None`.

If `None`, an empty dictionary is used.

Notes

The wrapper uses the default functions provided by Quantus: element-wise difference for comparing explanations, the Frobenius norm for the numerator and denominator, the default normalisation function and uniform-noise perturbations.

Raises

ValueError – If `explain_func` is not provided.

Parameters

- `context` (`MetricContext`)
- `explain_func` (`ExplainFunc`)
- `params` (`Mapping[str, Any] | None`)

NAME: `str = 'AvgSensitivity'`

`run()`

Compute the Average Sensitivity metric.

The method passes the selected inputs, target labels, original attributions and explanation function to `quantus.AvgSensitivity`. Quantus repeatedly perturbs each input, recomputes its explanation and averages the relative explanation changes across the configured number of samples.

If all attribution values are negative, their absolute values are used when `abs=True`; otherwise, the metric is skipped. The model is set to training mode and the device stored in the context is forwarded to Quantus.

Returns

Average Sensitivity score for each evaluated observation. Lower values indicate greater robustness to random input perturbations.

Return type

`List[float]`

Raises

MetricSkipped – If all attribution values are negative and `abs` is `False`.

3.1.5.2 Module contents

3.2 Submodules

3.2.1 `xai_metrics.metrics.autodiscover` module

`xai_metrics.metrics.autodiscover.autodiscover_metrics(package)`

Import all submodules of a metrics package.

This function recursively walks through `package` and imports every discovered module. Importing the modules triggers any registration logic executed at import time, such as `@register_metric`.

Parameters

package (*ModuleType*) – Imported package whose submodules should be discovered. The package must expose a `__path__` attribute.

Return type

None

Raises

- **AttributeError** – If `package` is not a package with a `__path__` attribute.
- **ImportError** – If a discovered submodule cannot be imported.

3.3 Module contents

XAI_METRICS.REPORTING PACKAGE

4.1 Submodules

4.1.1 xai_metrics.reporting.reporting module

`xai_metrics.reporting.reporting.build_reports(context_outputs)`

Build metric reports from context outputs.

Each context output must contain a `metadata` dictionary and a `results` dictionary. The metadata must define `dataset_name`, `model_name` and `xai_method_name`. An optional `metric_params` mapping may be associated with each metric with the parameters used during its evaluation.

Results are grouped by dataset and model. The resulting dataframe contains the fixed columns `dataset_name`, `model_name`, `metric` and `metric_params`, followed by one result column for each XAI method.

Numeric metric results are flattened, missing values are removed, and the remaining values are represented by their mean. Non-numeric values are serialized without aggregation. When a metric result is a mapping, each entry is expanded into an independent report row using the name "`<metric_name>.<result_key>`". All XAI methods evaluated for the same dataset and model must use identical parameters for a given report metric.

Parameters

context_outputs (*Sequence[Mapping[str, Any]]*) – Sequence of context output mappings. Each element must contain:

- **metadata** : Mapping[str, Any] Metadata identifying the dataset, model and XAI method.
- **results** : Mapping[str, Any] Results indexed by metric name.
- **metric_params** : Mapping[str, Any], optional Parameters used by each metric.

Returns

Nested report dictionary. The first level is indexed by dataset name and the second level by model name. Each value is a dataframe with one row per metric and one result column per XAI method.

Return type

Dict[str, Dict[str, pandas.DataFrame]]

Raises

- **KeyError** – If a context output does not contain the required metadata or result fields.
- **ValueError** – If more than one context output is provided for the same dataset, model and XAI method; if different parameter configurations are found for the same metric; or if an XAI method name conflicts with a reserved report column.

`xai_metrics.reporting.reporting.save_reports(reports, output_dir, report_name='xai_metrics_report')`

Save all metric reports as consolidated CSV and JSON files.

The report dataframes for every dataset and model are concatenated into a single report. Rows are sorted by dataset name, model name and metric name, while XAI method columns are sorted alphabetically.

Structured CSV cell values, such as metric parameter dictionaries, are encoded as JSON strings. The JSON file contains the complete report as a list of serialized record mappings under the "report" key.

Parameters

- **reports** (*Mapping[str, Mapping[str, pandas.DataFrame]]*) – Nested dictionary of report dataframes, usually returned by `build_reports()`. The first level must be indexed by dataset name and the second level by model name.
- **output_dir** (*str or pathlib.Path*) – Directory where the report files will be saved. The directory and its missing parent directories are created automatically.
- **report_name** (*str, default="xai_metrics_report"*) – Base filename used for the generated report files. The .csv and .json extensions are added automatically.

Returns

Dictionary containing the paths of the generated files under the keys "csv" and "json".

Return type

Dict[str, str]

Raises

ValueError – If reports does not contain any report dataframes to save.

4.2 Module contents

XAI_METRICS.RUNNER PACKAGE

5.1 Submodules

5.1.1 xai_metrics.runner.runner module

```
xai_metrics.runner.runner.run_evaluation(context=None, metadata=None, selected_metrics=None,  
                                         config='config.yaml',  
                                         report_output_dir=PosixPath('results/reports'),  
                                         **runtime_kwargs)
```

Run the XAI metric evaluation pipeline.

The function discovers all metric modules, loads the experiment configuration, builds or accepts one or more metric contexts, resolves the metrics that can be executed, injects runtime dependencies and evaluates each metric.

When `context` is `None`, contexts are built from `config` through `ConfigController`. When a context is supplied directly, `metadata` must also be provided.

Runtime keyword arguments are forwarded as dependencies to metric constructors when their names match constructor parameters. The special `model_loader` argument is also passed to `ConfigController`.

The special `explain_funcs` argument may contain a mapping from XAI method names to explanation functions. Method names are matched case-insensitively. The function corresponding to the current context is injected into compatible metrics under the `explain_func` dependency.

Successfully evaluated metrics are stored together with the parameters defined for them in the configuration. Metrics that raise `MetricSkipped` are recorded separately and do not stop the remaining evaluations.

Finally, the collected outputs are transformed into report dataframes. If `report_output_dir` is not `None`, the reports are also saved to disk.

Parameters

- **context** (`MetricContext` or `None`, optional) – Pre-built metric evaluation context. If `None`, one or more contexts are constructed from `config`.
- **metadata** (`Mapping[str, Any]` or `None`, optional) – Metadata associated with a directly supplied context. It must contain `dataset_name`, `model_name` and `xai_method_name`. Ignored when contexts are built from the configuration.
- **selected_metrics** (`Iterable[str]` or `None`, optional) – Metric names to execute. Only metrics that are also configured and registered are evaluated. If `None`, all configured and registered metrics are evaluated.
- **config** (`Mapping[str, Any]`, `str` or `pathlib.Path`, default="config.yaml") – Experiment configuration source. It may be a mapping or the path to a YAML configuration file.

- **report_output_dir** (*str*, *pathlib.Path* or *None*, *optional*) – Directory where the generated reports are saved. If *None*, reports are built but are not written to disk. The default directory is `results/reports`.
- ****runtime_kwargs** (*Any*) – Runtime dependencies made available to metric constructors. Supported values depend on the selected metrics and may include custom similarity, normalisation, perturbation or explanation functions.

Two arguments receive special handling:

- **model_loader** : Callable, optional Function used by the configuration controller to load models.
- **explain_funcs** : Mapping[*str*, Callable], optional Mapping from XAI method names to explanation functions. Method names are matched case-insensitively.

Returns

Dictionary containing the following entries:

- **"contexts"** : List[Dict[*str*, *Any*]] Raw output for each evaluated context. Every context output contains `metadata`, `results`, `metric_params` and `skipped`.
- **"reports"** : Dict[*str*, Dict[*str*, pandas.DataFrame]] Report dataframes grouped first by dataset name and then by model name.
- **"report_paths"** : Dict[*str*, *str*] Paths of the saved report files. The dictionary is empty when `report_output_dir` is *None*.

Return type

Dict[*str*, *Any*]

Raises

- **ValueError** – If metadata required for a directly supplied context is missing, if no explanation function is available for the current XAI method, or if the configuration or report data is invalid.
- **TypeError** – If context construction loads an object that is not a supported model type.
- **RuntimeError** – If context construction requests a device that is not available.

Warns

UserWarning – If configured or selected metrics cannot be executed because they are missing from the configuration or metric registry.

5.2 Module contents

PYTHON MODULE INDEX

X

`xai_metrics.base`, 4
`xai_metrics.base.base_explainer`, 1
`xai_metrics.base.base_metric`, 2
`xai_metrics.base.registry`, 3
`xai_metrics.base.types`, 4
`xai_metrics.config`, 6
`xai_metrics.config.config_controller`, 5
`xai_metrics.metrics`, 29
`xai_metrics.metrics.autodiscover`, 28
`xai_metrics.metrics.complexity`, 9
`xai_metrics.metrics.complexity.complexity_metric`, 7
`xai_metrics.metrics.complexity.sparseness`, 8
`xai_metrics.metrics.faithfulness`, 18
`xai_metrics.metrics.faithfulness.consistency`, 9
`xai_metrics.metrics.faithfulness.faithfulness`, 10
`xai_metrics.metrics.faithfulness.faithfulness_estimate`, 11
`xai_metrics.metrics.faithfulness.monotonicity`, 12
`xai_metrics.metrics.faithfulness.monotonicity_correlation`, 13
`xai_metrics.metrics.faithfulness.monotonicity_metric`, 15
`xai_metrics.metrics.faithfulness.sensitivity_n`, 16
`xai_metrics.metrics.faithfulness.sufficiency`, 17
`xai_metrics.metrics.fidelity`, 21
`xai_metrics.metrics.fidelity.completeness`, 20
`xai_metrics.metrics.fidelity.completeness.completeness_metric`, 18
`xai_metrics.metrics.fidelity.soundness`, 21
`xai_metrics.metrics.fidelity.soundness.non_sensitivity`, 20
`xai_metrics.metrics.robustness`, 27
`xai_metrics.metrics.robustness.local_lipschitz_estimate`, 21
`xai_metrics.metrics.robustness.max_sensitivity`, 23
`xai_metrics.metrics.robustness.relative_input_stability`, 24
`xai_metrics.metrics.robustness.relative_output_stability`, 25
`xai_metrics.metrics.sensitivity`, 28
`xai_metrics.metrics.sensitivity.avg_sensitivity`, 27
`xai_metrics.reporting`, 32
`xai_metrics.reporting.reporting`, 31
`xai_metrics.runner`, 34
`xai_metrics.runner.runner`, 33

INDEX

A

`as_explain_func()` (*xai_metrics.base.base_explainer.BaseExplainer* method), 1

`autodiscover_metrics()` (in module *xai_metrics.metrics.autodiscover*), 28

`AvgSensitivity` (class in *xai_metrics.metrics.sensitivity.avg_sensitivity*), 27

B

`BaseExplainer` (class in *xai_metrics.base.base_explainer*), 1

`BaseMetric` (class in *xai_metrics.base.base_metric*), 2

`build_context()` (*xai_metrics.config.config_controller.ConfigController* method), 5

`build_contexts()` (*xai_metrics.config.config_controller.ConfigController* method), 6

`build_metrics_from_config()` (in module *xai_metrics.base.registry*), 3

`build_reports()` (in module *xai_metrics.reporting.reporting*), 31

C

`Completeness` (class in *xai_metrics.metrics.fidelity.completeness.completeness_metric*), 18

`Complexity` (class in *xai_metrics.metrics.complexity.complexity_metric*), 7

`ConfigController` (class in *xai_metrics.config.config_controller*), 5

`Consistency` (class in *xai_metrics.metrics.faithfulness.consistency*), 9

D

`default_model_loader()` (in module *xai_metrics.config.config_controller*), 6

`device` (*xai_metrics.base.base_metric.MetricContext* attribute), 3

E

`explain()` (*xai_metrics.base.base_explainer.BaseExplainer* method), 1

`ExplainFunc` (in module *xai_metrics.base.types*), 4

F

`Faithfulness` (class in *xai_metrics.metrics.faithfulness.faithfulness*), 10

`FaithfulnessEstimate` (class in *xai_metrics.metrics.faithfulness.faithfulness_estimate*), 11

G

`get_metrics_config()` (*xai_metrics.config.config_controller.ConfigController* method), 6

L

`list_metrics()` (in module *xai_metrics.base.registry*), 3

`LocalLipschitzEstimate` (class in *xai_metrics.metrics.robustness.local_lipschitz_estimate*), 21

M

`MaxSensitivity` (class in *xai_metrics.metrics.robustness.max_sensitivity*), 23

`MetricContext` (class in *xai_metrics.base.base_metric*), 2

`MetricSkipped`, 3

module

- xai_metrics.base*, 4
- xai_metrics.base.base_explainer*, 1
- xai_metrics.base.base_metric*, 2
- xai_metrics.base.registry*, 3
- xai_metrics.base.types*, 4
- xai_metrics.config*, 6
- xai_metrics.config.config_controller*, 5
- xai_metrics.metrics*, 29
- xai_metrics.metrics.autodiscover*, 28
- xai_metrics.metrics.complexity*, 9
- xai_metrics.metrics.complexity.complexity_metric*, 7

```

xai_metrics.metrics.complexity.sparseness, 8
xai_metrics.metrics.faithfulness, 18
xai_metrics.metrics.faithfulness.consistency, 9
xai_metrics.metrics.faithfulness.faithfulness, 10
xai_metrics.metrics.faithfulness.faithfulness_estimate, 11
xai_metrics.metrics.faithfulness.monotonicity, 12
xai_metrics.metrics.faithfulness.monotonicity_correlation, 13
xai_metrics.metrics.faithfulness.monotonicity_metric, 15
xai_metrics.metrics.faithfulness.sensitivity_n, 16
xai_metrics.metrics.faithfulness.sufficiency, 17
xai_metrics.metrics.fidelity, 21
xai_metrics.metrics.fidelity.completeness, 20
xai_metrics.metrics.fidelity.completeness.completeness_metric, 18
xai_metrics.metrics.fidelity.soundness, 21
xai_metrics.metrics.fidelity.soundness.non_sensitivity, 20
xai_metrics.metrics.robustness, 27
xai_metrics.metrics.robustness.local_lipschitz_estimate, 21
xai_metrics.metrics.robustness.max_sensitivity, 23
xai_metrics.metrics.robustness.relative_input_stability, 24
xai_metrics.metrics.robustness.relative_output_stability, 25
xai_metrics.metrics.sensitivity, 28
xai_metrics.metrics.sensitivity.avg_sensitivity, 27
xai_metrics.reporting, 32
xai_metrics.reporting.reporting, 31
xai_metrics.runner, 34
xai_metrics.runner.runner, 33

Monotonicity (class in
    xai_metrics.metrics.faithfulness.monotonicity), 12
MonotonicityCorrelation (class in
    xai_metrics.metrics.faithfulness.monotonicity_correlation), 13
MonotonicityMetric (class in
    xai_metrics.metrics.faithfulness.monotonicity_metric), 15

NAME (xai_metrics.base.base_explainer.BaseExplainer
    attribute), 1
NAME (xai_metrics.base.base_metric.BaseMetric
    attribute), 2
NAME (xai_metrics.metrics.complexity.complexity_metric.Complexity
    attribute), 7
NAME (xai_metrics.metrics.complexity.sparseness.Sparseness
    attribute), 8
NAME (xai_metrics.metrics.faithfulness.consistency.Consistency
    attribute), 9
NAME (xai_metrics.metrics.faithfulness.faithfulness.Faithfulness
    attribute), 10
NAME (xai_metrics.metrics.faithfulness.faithfulness_estimate.FaithfulnessEst
    attribute), 12
NAME (xai_metrics.metrics.faithfulness.monotonicity.Monotonicity
    attribute), 13
NAME (xai_metrics.metrics.faithfulness.monotonicity_correlation.Monotonic
    attribute), 14
NAME (xai_metrics.metrics.faithfulness.monotonicity_metric.MonotonicityM
    attribute), 15
NAME (xai_metrics.metrics.faithfulness.sensitivity_n.SensitivityN
    attribute), 16
NAME (xai_metrics.metrics.faithfulness.sufficiency.Sufficiency
    attribute), 17
NAME (xai_metrics.metrics.fidelity.completeness.completeness_metric.Comp
    attribute), 19
NAME (xai_metrics.metrics.fidelity.soundness.non_sensitivity.NonSensitivity
    attribute), 21
NAME (xai_metrics.metrics.robustness.local_lipschitz_estimate.LocalLipsch
    attribute), 22
NAME (xai_metrics.metrics.robustness.max_sensitivity.MaxSensitivity
    attribute), 24
NAME (xai_metrics.metrics.robustness.relative_input_stability.RelativeInput
    attribute), 25
NAME (xai_metrics.metrics.robustness.relative_output_stability.RelativeOutp
    attribute), 26
NAME (xai_metrics.metrics.sensitivity.avg_sensitivity.AvgSensitivity
    attribute), 28
NonSensitivity (class in
    xai_metrics.metrics.fidelity.soundness.non_sensitivity), 20

R
register_metric() (in module
    xai_metrics.base.registry), 4
RelativeInputStability (class in
    xai_metrics.metrics.robustness.relative_input_stability), 24
RelativeOutputStability (class in
    xai_metrics.metrics.robustness.relative_output_stability), 25
run() (xai_metrics.base.base_metric.BaseMetric
    method), 2

```

`run()` (`xai_metrics.metrics.complexity.complexity_metric.ComplexityMetric`, 2
 `method`), 7 `xai_metrics.base.registry`
`run()` (`xai_metrics.metrics.complexity.sparseness.Sparseness` `module`, 3
 `method`), 8 `xai_metrics.base.types`
`run()` (`xai_metrics.metrics.faithfulness.consistency.Consistency` `module`, 4
 `method`), 9 `xai_metrics.config`
`run()` (`xai_metrics.metrics.faithfulness.faithfulness.Faithfulness` `module`, 6
 `method`), 10 `xai_metrics.config.config_controller`
`run()` (`xai_metrics.metrics.faithfulness.faithfulness_estimate.FaithfulnessEstimate`
 `method`), 12 `xai_metrics.metrics`
`run()` (`xai_metrics.metrics.faithfulness.monotonicity.Monotonicity` `module`, 29
 `method`), 13 `xai_metrics.metrics.autodiscover`
`run()` (`xai_metrics.metrics.faithfulness.monotonicity_correlation.MonotonicityCorrelation`
 `method`), 14 `xai_metrics.metrics.complexity`
`run()` (`xai_metrics.metrics.faithfulness.monotonicity_metric.MonotonicityMetric`
 `method`), 15 `xai_metrics.metrics.complexity.complexity_metric`
`run()` (`xai_metrics.metrics.faithfulness.sensitivity_n.SensitivityN` `module`, 7
 `method`), 16 `xai_metrics.metrics.complexity.sparseness`
`run()` (`xai_metrics.metrics.faithfulness.sufficiency.Sufficiency` `module`, 8
 `method`), 17 `xai_metrics.metrics.faithfulness`
`run()` (`xai_metrics.metrics.fidelity.completeness.completeness` `module`, 18
 `method`), 19 `xai_metrics.metrics.faithfulness.consistency`
`run()` (`xai_metrics.metrics.fidelity.soundness.non_sensitivity.NonSensitivity`
 `method`), 21 `xai_metrics.metrics.faithfulness.faithfulness`
`run()` (`xai_metrics.metrics.robustness.local_lipschitz_estimate.LocalLipschitzEstimate`
 `method`), 22 `xai_metrics.metrics.faithfulness.faithfulness_estimate`
`run()` (`xai_metrics.metrics.robustness.max_sensitivity.MaxSensitivity` `module`, 11
 `method`), 24 `xai_metrics.metrics.faithfulness.monotonicity`
`run()` (`xai_metrics.metrics.robustness.relative_input_stability.RelativeInputStability`
 `method`), 25 `xai_metrics.metrics.faithfulness.monotonicity_correlation`
`run()` (`xai_metrics.metrics.robustness.relative_output_stability.RelativeOutputStability`
 `method`), 26 `xai_metrics.metrics.faithfulness.monotonicity_metric`
`run()` (`xai_metrics.metrics.sensitivity.avg_sensitivity.AvgSensitivity` `module`, 15
 `method`), 28 `xai_metrics.metrics.faithfulness.sensitivity_n`
`run_evaluation()` (`in` `module` `module`, 16
 `xai_metrics.runner.runner`), 33 `xai_metrics.metrics.faithfulness.sufficiency`
 `module`, 17
S `xai_metrics.metrics.fidelity`
`save_reports()` (`in` `module` `module`, 21
 `xai_metrics.reporting.reporting`), 31 `xai_metrics.metrics.fidelity.completeness`
`SensitivityN` (`class` `in` `module`, 20
 `xai_metrics.metrics.faithfulness.sensitivity_n`), `xai_metrics.metrics.fidelity.completeness.completeness_metric`
 16 `module`, 18
`Sparseness` (`class in xai_metrics.metrics.complexity.sparseness`, `xai_metrics.metrics.fidelity.soundness`
 8 `module`, 21
`Sufficiency` (`class` `in` `xai_metrics.metrics.fidelity.soundness.non_sensitivity`
 `xai_metrics.metrics.faithfulness.sufficiency`), 17 `module`, 20
 `xai_metrics.metrics.robustness`
 `module`, 27
X `xai_metrics.metrics.robustness.local_lipschitz_estimate`
`xai_metrics.base` `module`, 4 `module`, 21
`xai_metrics.base.base_explainer` `xai_metrics.metrics.robustness.max_sensitivity`
 `module`, 1 `module`, 23
`xai_metrics.base.base_metric` `xai_metrics.metrics.robustness.relative_input_stability`

- module, [24](#)
- xai_metrics.metrics.robustness.relative_output_stability
 - module, [25](#)
- xai_metrics.metrics.sensitivity
 - module, [28](#)
- xai_metrics.metrics.sensitivity.avg_sensitivity
 - module, [27](#)
- xai_metrics.reporting
 - module, [32](#)
- xai_metrics.reporting.reporting
 - module, [31](#)
- xai_metrics.runner
 - module, [34](#)
- xai_metrics.runner.runner
 - module, [33](#)